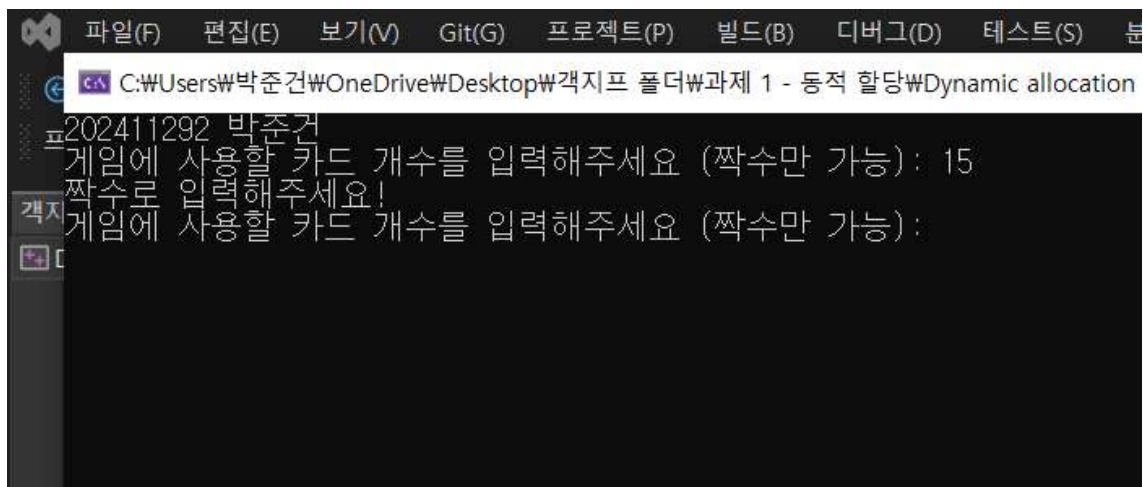


## 20241129 박준건 객지프 과제 보고서

### 1. 게임 보드 초기화 및 동적 메모리 할당

```
do {  
    cout << "게임에 사용할 카드 개수를 입력해주세요 (짝수만 가능): ";  
    cin >> cardCount;  
    if (cardCount % 2 != 0) {  
        cout << "짝수로 입력해주세요!" << endl;  
    }  
} while (cardCount % 2 != 0);
```

짝수의 입력이 들어오지 않았을 시엔 do - while 무한루프를 통해 처리하도록 하였습니다.



```
width = 1;  
height = cardCount;  
for (int i = 1; i <= sqrt(cardCount); ++i) {  
    if (cardCount % i == 0) {  
        int j = cardCount / i;  
        if (abs(i - j) < abs(width - height)) {  
            width = i;  
            height = j;  
        }  
    }  
}
```

정사각형에 가깝도록 정하는 문제의 경우, 정사각형과 비슷한 도형의 특징인 '변끼리의 차이가 작다'라는 점을 이용했습니다. 카드의 제곱근보다 작은 값들 중에서 카드 수의 약수 두 개를 찾아낸 후에, 그 차이가 제일 적은 것을 찾기 위해 for문을 돌렸습니다. 결과적으로 서로간의 차이가 제일

적은 cardcount의 약수 두 개가 선택됩니다.

```
C:\Users\박준건\OneDrive\Desktop\객지프 폴더\과제 1 - 동적 할당\Dynamic allocation -
202411292 박준건
게임에 사용할 카드 개수를 입력해주세요 (짝수만 가능): 15
짝수로 입력해주세요!
게임에 사용할 카드 개수를 입력해주세요 (짝수만 가능): 18

[현재 보드 상태]
*      *      *      *      *      *
*      *      *      *      *      *
*      *      *      *      *      *

첫 번째 카드 좌표 입력 (행 열):
```

```
C:\Users\박준건\OneDrive\Desktop\객지프 폴더\과제 1 - 동적 할당\Dynamic allocatio
202411292 박준건
게임에 사용할 카드 개수를 입력해주세요 (짝수만 가능): 20

[현재 보드 상태]
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *

첫 번째 카드 좌표 입력 (행 열):
```

```
int value = 1;
int count = 0;
for (int i = 0; i < width; ++i) {
    for (int j = 0; j < height; ++j) {
        board[i][j] = value;
        count++;
        if (count == 2) {
            value++;
            count = 0;
        }
    }
}
```

```

    }
}

```

```
shuffleCard(board, width, height);
```

보드엔 숫자 값을 채워넣도록 하였으며, 한 숫자가 두 번 이상 채워진다면 count가 2가 되어서 다음 숫자로 넘어가게 됩니다. 이후 숫자가 채워진 배열을 셔플해줍니다.

```

for (int i = total - 1; i > 0; --i) {
    int j = rand() % (i + 1);
    int tempVal = temp[i];
    temp[i] = temp[j];
    temp[j] = tempVal;
}

```

카드 셔플은 전통적인 랜덤 값 결정 -> 지금 값과 서로 변경하는 알고리즘을 사용하였습니다.

## 2. 게임 진행 로직 / 3. 입력 예외 처리.

```

bool** opened = new bool* [width];
for (int i = 0; i < width; ++i)
    opened[i] = new bool[height] {false};

```

맞춘 여부(open)는 open 타입의 bool 배열을 선언해주어서 체크하도록 하였습니다.

```

if (x1 < 0 || x1 >= width || y1 < 0 || y1 >= height ||
    x2 < 0 || x2 >= width || y2 < 0 || y2 >= height) {
    cout << "보드 범위를 벗어났습니다. 다시 입력해주세요.\n";
    continue;
}

```

```

if (opened[x1][y1] || opened[x2][y2]) {
    cout << "이미 맞춘 카드입니다. 다시 선택해주세요.\n";
    continue;
}

```

범위를 벗어나거나 한번 열어봤던 카드의 경우 범위 검사를 통해 걸러집니다.

```

// 메모리 해제
C:\Users\박준건\OneDrive\Desktop\객지프 폴더\과제 1 - 등적 할당\Dynamic a

[현재 보드 상태]
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *

첫 번째 카드 좌표 입력 (행 열): 4 0
두 번째 카드 좌표 입력 (행 열): 4 0
보드 범위를 벗어났습니다. 다시 입력해주세요.

[현재 보드 상태]
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *
*      *      *      *      *

첫 번째 카드 좌표 입력 (행 열):

```

```

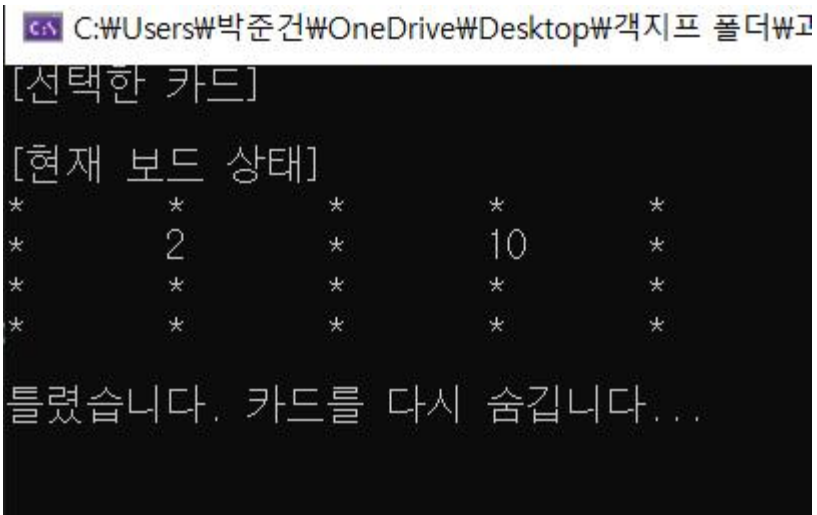
system("cls");
cout << "[선택한 카드]\n";
printBoard(board, opened, width, height, x1, y1, x2, y2);

if (board[x1][y1] == board[x2][y2]) {
    cout << "\n짝을 맞췄습니다!\n";
    opened[x1][y1] = true;
    opened[x2][y2] = true;
    matchedPairs++;
}
else {
    cout << "\n틀렸습니다. 카드를 다시 숨깁니다...\n";
}

Sleep(2000);
system("cls");
}

```

짝을 맞추든 틀렸든, 선택한 카드 내부의 값은 모두에게 보여집니다. sleep 함수를 이용해서 2초만 큼 기다리도록 했습니다.



```
void printBoard(int** board, bool** opened, int width, int height, int x1, int y1, int x2,
int y2) {
    cout << "\n[현재 보드 상태]\n";
    for (int i = 0; i < width; ++i) {
        for (int j = 0; j < height; ++j) {
            if (opened[i][j] || (i == x1 && j == y1) || (i == x2 && j == y2))
                cout << board[i][j] << "\t";
            else
                cout << "*\t";
        }
        cout << endl;
    }
}
```

printBoard 함수에선 선택한 카드의 값을 출력하도록 합니다.

## 5. 메모리 관리

```
// 메모리 해제
for (int i = 0; i < width; ++i)
    delete[] opened[i];
delete[] opened;

void freeMemory(int**& memory, int width) {
    for (int i = 0; i < width; ++i)
        delete[] memory[i];
}
```

```

    delete[] memory;
    memory = nullptr;
}

```

동적 할당으로 사용하였던 메모리의 경우엔 모두 끝에 메모리 할당을 해제하는 과정을 밟았습니다.

```

#define _CRTDBG_MAP_ALLOC
#ifdef _DEBUG
#ifndef DBG_NEW
#define DBG_NEW new ( _NORMAL_BLOCK , __FILE__ , __LINE__ )
#define new DBG_NEW
#endif // !DBG_NEW
#endif

```

```

_CrtDumpMemoryLeaks();

```

또한 해당 코드를 추가해서 메모리 누출을 검사할 수 있도록 하였습니다.

```

'Dynamic allocation - PA1.exe'(Win32): 'C:\Windows\System32\kernel.appcore.dll'을(를) 로
'Dynamic allocation - PA1.exe'(Win32): 'C:\Windows\System32\msvcrt.dll'을(를) 로드했습니
25768 스레드가 종료되었습니다(코드: 0 (0x0)).
23140 스레드가 종료되었습니다(코드: 0 (0x0)).
24236 스레드가 종료되었습니다(코드: 0 (0x0)).
'[31300] Dynamic allocation - PA1.exe' 프로그램이 종료되었습니다(코드: 0 (0x0)).

```